Universidad Nacional de Córdoba
Facultad de Ciencias Exactas, Físicas y Naturales

Sistemas de Computación
Trabajo Práctico N° 2

Docentes:
Ing. Miguel Ángel Solinas
Ing. Javier Alejandro Jorge

Autores:
Matías Lucas Morales Robles
José Ignacio Torres
Esteban Samuel Suarez

Índice

1. Introducción **2**

2. Desarrollo **3**
2.1. Primera Iteración 3
2.2. Segunda Iteración 5

3. Análisis de la Dinámica del Stack en la Ejecución de la Subrutina `convert` **8**
3.1. Antes de la llamada a `convert` 8
3.2. Estado de la subrutina `convert` tras el ingreso al flujo de ejecución, pero previo al push del registro `rbp` 8
3.3. Luego de ejecutar `push rbp` 9
3.4. Algunas instrucciones que no afectan al stack 9
3.5. Ejecución de `pop rbp` 10
3.6. Ejecución de `ret` (retornar a `convertion`) 10

4. Conclusión **11**

5. Bibliografía **11**

1. Introducción

En los sistemas modernos que integran hardware y software, es habitual emplear arquitecturas en capas para abordar el desarrollo de aplicaciones complejas. Este enfoque permite asignar responsabilidades específicas a cada nivel, favoreciendo tanto la organización como la eficiencia del sistema.

En las capas superiores, se prioriza el uso de lenguajes de alto nivel, los cuales resultan más adecuados para la implementación de interfaces de usuario (UI) y la interacción con servicios externos. En contraposición, las capas inferiores se orientan a tareas más cercanas al hardware, donde se requiere mayor control y rendimiento.

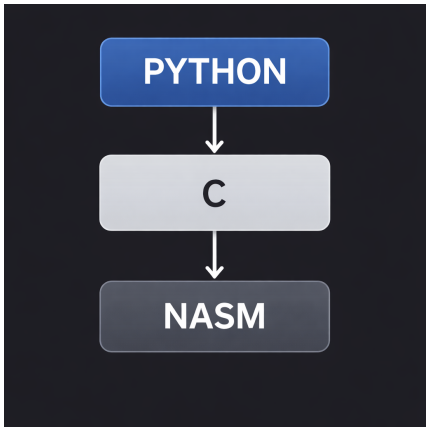
En el presente trabajo se adopta esta filosofía mediante la integración de múltiples lenguajes de programación, cada uno ubicado en una capa distinta. En primer lugar, se utilizará Python para desarrollar la interfaz de usuario y realizar consultas a la API de WorldBank, específicamente para obtener datos relacionados con el índice de GINI.

A continuación, en una segunda capa, se empleará el lenguaje C con el objetivo de procesar dichos datos de manera más eficiente, aprovechando su cercanía al hardware y su mejor desempeño en operaciones de bajo nivel en comparación con Python.

Finalmente, en una tercera capa aún más próxima al hardware, se utilizará Netwide Assembler (NASM) para llevar a cabo el procesamiento matemático final. Este enfoque permite evidenciar cómo los lenguajes de bajo nivel pueden ofrecer ventajas significativas en términos de rendimiento, especialmente en contextos donde los tiempos de ejecución son críticos.

Cabe destacar que los lenguajes de bajo nivel, como el ensamblador, surgieron como una primera abstracción sobre el lenguaje de máquina, permitiendo una programación más accesible sin perder el control directo sobre la arquitectura. Por su parte, los lenguajes de alto nivel dependen de estos niveles inferiores para interactuar con el hardware, lo cual se logra mediante convenciones de llamada.

En este contexto, comprender dichas convenciones resulta fundamental, ya que permiten la correcta comunicación entre funciones escritas en distintos lenguajes, facilitando la integración de todo el sistema. Este conocimiento es especialmente relevante en áreas como el desarrollo de sistemas críticos, la seguridad informática y el estudio profundo de la interacción entre software y hardware.



2. Desarrollo

2.1. Primera Iteración

En la primera iteración del trabajo, se obtienen datos correspondientes al índice GINI de Argentina a lo largo de distintos años mediante consultas a la API de WorldBank. Dichos datos son recibidos en formato `JSON` y procesados inicialmente utilizando Python, un lenguaje de alto nivel adecuado para este tipo de tareas.

Posteriormente, Python invoca una función implementada en lenguaje C, representando así una capa inferior dentro de la arquitectura planteada. Esta función tiene como objetivo realizar dos operaciones sobre los datos recibidos: por un lado, convertir los valores de tipo flotante a enteros, y por otro, sumar una unidad a cada elemento como forma de validación del procesamiento realizado.

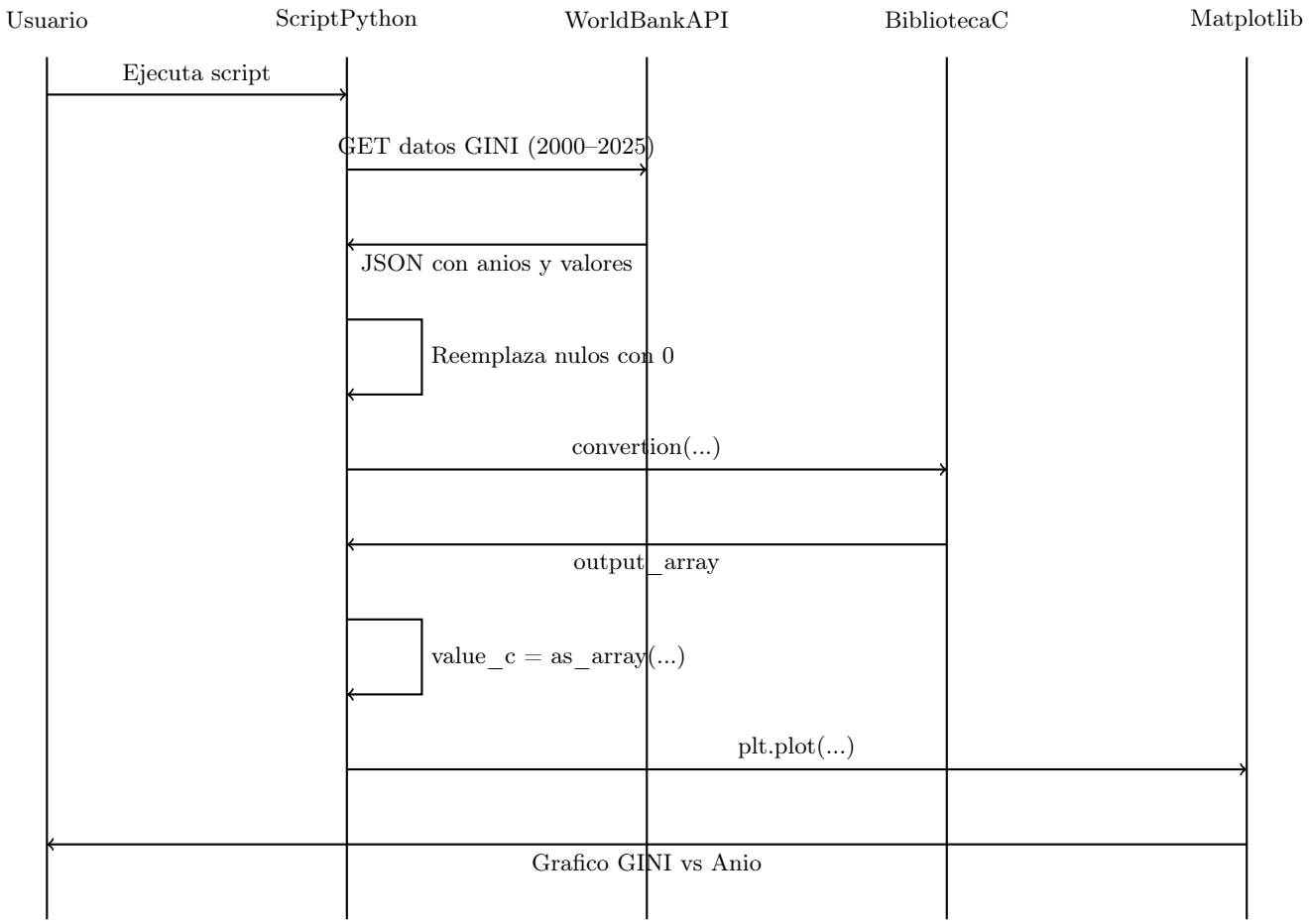
- Conversión de valores de punto flotante a enteros.
- Incremento en una unidad de cada valor procesado.

Para lograr la comunicación entre Python y C, se emplea la biblioteca `ctypes`, la cual permite cargar dinámicamente una librería compartida (en este caso, `main.so`) generada a partir de la compilación del código en C. Esta metodología es equivalente al uso de una biblioteca de funciones externas.

Un aspecto relevante durante la implementación fue la gestión de los parámetros entre ambos lenguajes. La función en C recibe como entrada punteros a arreglos de valores de punto flotante, lo cual presenta ciertas dificultades al ser manejado desde Python debido a su tipado dinámico. Sin embargo, mediante `ctypes`, es posible definir explícitamente tanto los tipos de los parámetros como el tipo de retorno de la función, permitiendo así una correcta interoperabilidad entre ambos lenguajes.

```python
def convertion(input, output, length):
    main_c.convertion(input, output, length)

lib_path = os.path.join(os.path.dirname(__file__), 'main.so')
main_c = ctypes.CDLL(lib_path)

main_c.convertion.argtypes = (
    ctypes.POINTER(ctypes.c_float),
    ctypes.POINTER(ctypes.c_int),
    ctypes.c_int
)
main_c.convertion.restype = ctypes.c_void_p
```

A continuación se muestra el diagrama de secuencia del script GINI



2.2. Segunda Iteración

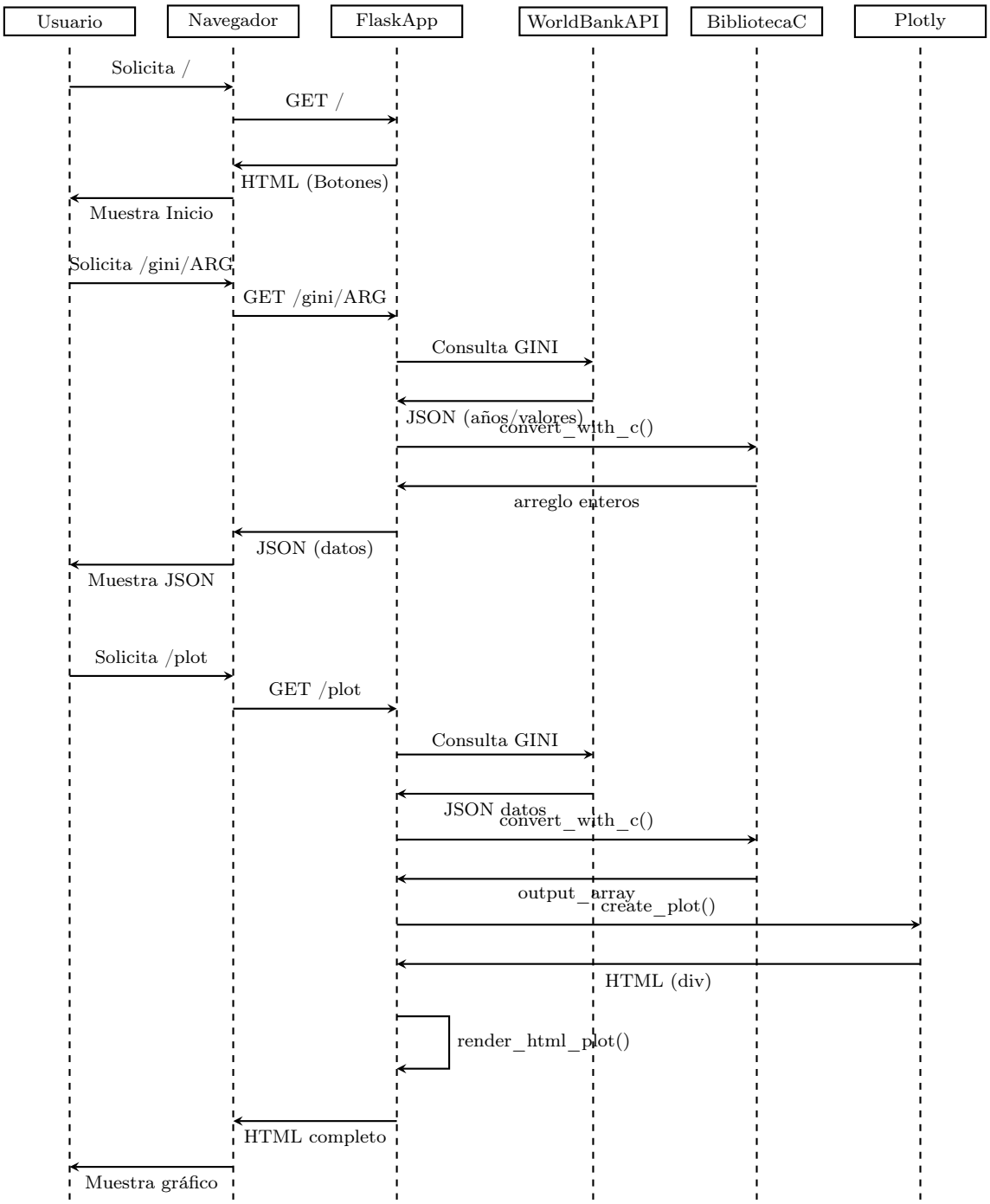
En esta segunda iteración del trabajo se incorpora una capa adicional aún más cercana al hardware, delegando el cálculo final al lenguaje ensamblador mediante el uso de *Netwide Assembler* (NASM). En este contexto, se aplican convenciones de llamada para garantizar una correcta interoperabilidad entre los distintos niveles del sistema.

Por otro lado, la interfaz de usuario es migrada a una aplicación web local desarrollada con *Flask* en Python. Esta permite, a través de una petición `GET`, obtener el gráfico correspondiente a los datos procesados, los cuales ya han atravesado las capas inferiores de procesamiento. La selección de los datos se realiza mediante un identificador de país denominado `Country_Code`, basado en el estándar ISO 3166-1 alpha-3.

De esta manera, se logra una integración completa entre las distintas capas del sistema, manteniendo una clara separación de responsabilidades. A continuación, se presenta la forma en la cual se realiza la consulta a la API y el preprocesamiento de los datos antes de ser enviados a la capa implementada en C.

```python
def get_data():
    response = requests.get(
        "https://api.worldbank.org/v2/en/country/ARG/indicator/"
        "SI.POV.GINI?format=json&date=2000:2025"
    )

    if response.ok:
        data = response.json()
        results = data[1]  # Second element contains the data

        year = []
        value = []
        for entry in results:
            # Replace None values with 0
            year.append(entry['date'])
            if entry['value'] is not None:
                value.append(entry['value'])
            else:
                value.append(0)

        year = np.flip(np.array(year, dtype=float))
        value = np.flip(np.array(value, dtype=float))

        return year, value
    else:
        print("Failed to fetch data:", response.status_code)
        return None, None
```

A continuación se muestra el diagrama de secuencia completo de la app GINI:



Esta arquitectura permite el despliegue de un **servidor local** con una interfaz dinámica para la visualización de los índices. El flujo de información atraviesa las distintas **capas del sistema**, garantizando un procesamiento modular y eficiente de los datos desde su obtención hasta su representación gráfica.

3. Análisis de la Dinámica del Stack en la Ejecución de la Subrutina `convert`

3.1. Antes de la llamada a `convert`

Para esto, se hizo un breakpoint en `convertion`:

```
B+   17 void convertion(float* input, int* output, int length) {
     18     for (int i = 0; i < length; i++) {
     19         output[i] = convert(input[i]);
     20     }
     21 }
     22
     23 int main(void) {
     24     float  in[1]  = {38.5f};
     25     int    out[1] = {0};
     26
     27     convertion(in, out, 1);
     28     for (int i = 0; i < 1; i++)
     29         printf("in[%d]=%f → out[%d]=%d\n", i, in[i], i, out[i]);
     30
     31     return 0;
     32 }
```

El flujo de ejecución se encuentra actualmente dentro de la función `convertion`, previa a la invocación de la subrutina `convert`. En esta arquitectura (**x86_64**), los parámetros hacia `convertion` se reciben a través de registros (`RDI`, `RSI`, `RDX`) de acuerdo con la convención de llamadas *System V ABI*.

Por lo tanto, el *stack* está compuesto por la dirección de retorno hacia `main` y el espacio reservado para las variables locales de `convertion`. Al tratarse de un sistema de 64 bits, cada entrada en la pila tiene un tamaño de 8 bytes.

Dirección	Contenido	Descripción
`[rsp]`	Dirección de retorno	Retorno a `main` (tras finalizar `convertion`)
`[rsp+8]`	Variables locales	Espacio de datos de la función `convertion`

Entonces, en este punto el stack se ve de la siguiente manera:

```
(gdb) x/8x $rsp
0x7fffffffddb0: 0x000000000000c000      0x00000001ffffdde8
0x7fffffffddc0: 0x00007fffffffde14      0x00007fffffffde10
```

3.2. Estado de la subrutina `convert` tras el ingreso al flujo de ejecución, pero previo al push del registro `rbp`

Análisis: Ingreso a la subrutina `convert` (Post-CALL)

Una vez iniciada la ejecución de `convert`, el *stack* se organiza siguiendo la convención **System V AMD64 ABI**. En este instante, la instrucción `CALL` ya ha sido ejecutada, lo que ha modificado la pila para almacenar la dirección de retorno.

El stack en este punto se ve de la siguiente manera:

```
(gdb) x/8x $rsp
0x7fffffffdda8: 0x000055555555519f      0x000000000000c000
0x7fffffffddb8: 0x00000001ffffdde8      0x00007fffffffde14
```

Estado del sistema:

- **Dirección de retorno:** El valor `0x000055555555519f` se encuentra en la cima de la pila; es la dirección a la que regresará el flujo una vez finalizada la subrutina.
- **Parámetros:** A diferencia de la convención *cdecl*, el argumento `float` no se pasa por la pila, sino a través del registro `XMM0`.
- **Marco de pila:** Aún no se ha preservado el registro `RBP`. Esto ocurrirá en la siguiente instrucción (`push rbp`).

Dirección	Contenido	Descripción
`0x7fffffffdda8`	`0x000055555555519f`	Dirección de retorno (cima del stack)
`0x7fffffffddb0`	`0x000000000000c000`	Inicio del marco de `convertion`

3.3. Luego de ejecutar `push rbp`

Análisis: Ejecución de `push rbp` en `convert`

En esta instancia, se ha ejecutado la instrucción `push rbp` como primera sentencia de la subrutina `convert`. Esta acción inicia la creación del nuevo marco de pila (*stack frame*) siguiendo el estándar de **x86_64**, quedando la pila de la siguiente manera:

```
(gdb) x/8x $rsp
0x7fffffffdda0: 0x00007fffffffddf0      0x000055555555519f
0x7fffffffddb0: 0x000000000000c000      0x00000001ffffdde8
```

Estado del sistema:

- **Preservación del contexto:** El valor del registro `RBP` perteneciente a la función llamadora (`convertion`) ha sido almacenado en la pila. Esto permitirá restaurar el estado previo una vez que la subrutina finalice.
- **Movimiento del RSP:** Al realizar un `push` de un registro de 64 bits, el puntero `RSP` se decrementó en 8 bytes, pasando de `...dda8` a `...dda0`.
- **Ubicación de datos:** La dirección de retorno ha quedado desplazada una posición por debajo de la cima actual del *stack*.

Dirección	Contenido	Descripción
`0x7fffffffdda0`	`0x00007fffffffddf0`	Valor anterior de `RBP` (Salvado)
`0x7fffffffdda8`	`0x000055555555519f`	Dirección de retorno a `convertion`
`0x7fffffffddb0`	`0x000000000000c000`	Datos de contexto de la función previa

3.4. Algunas instrucciones que no afectan al stack

Análisis: Configuración del marco de pila y lógica interna

Tras la preservación del registro de base, se ejecutan instrucciones que definen la estructura del marco de pila y gestionan los datos, sin necesariamente alterar la cima del *stack* (`RSP`).

Detalles de la ejecución en x86_64:

- **Creación del marco de pila:** La instrucción `mov rbp, rsp` establece el inicio del nuevo *stack frame*. A partir de este momento, `RBP` se utiliza como dirección de referencia fija.
- **Acceso a argumentos:** A diferencia de *cdecl*, el argumento no se encuentra en `[rbp+16]`. En esta arquitectura, el valor `float` se recibe en el registro `XMM0` y, de ser necesario, se almacena en el espacio de variables locales (por ejemplo, en `[rbp-4]`).

- **Gestión de retorno:** El resultado de la conversión se almacena en el registro `RAX` (o `XMM0` si el retorno es un punto flotante), cumpliendo con la convención estándar.
- **Destrucción del marco:** La instrucción `mov rsp, rbp` (o simplemente `leave`) iguala el puntero de pila con el de base, invalidando el espacio de variables locales y preparando el retorno.

3.5. Ejecución de `pop rbp`

Análisis: Restauración del contexto (`pop rbp`)

Tras finalizar la lógica principal de la subrutina, se ejecuta la instrucción `pop rbp`. Esta operación es fundamental para desmantelar el marco de pila actual y restaurar el estado de la función llamadora.

Efectos de la instrucción:

- **Restauración de RBP:** El valor almacenado en la cima de la pila (el `RBP` antiguo) es extraído y cargado nuevamente en el registro `RBP`. Esto restablece el marco de pila de la función `convertion`.
- **Movimiento del RSP:** Al realizar el `pop`, el puntero de pila `RSP` se incrementa en 8 bytes, apuntando ahora directamente a la dirección de retorno.
- **Estado del Stack:** Como se observa en la captura de GDB, la cima de la pila vuelve a la dirección `...dda8`, donde reside la dirección de retorno hacia la lógica de C.

Dirección	Contenido	Descripción
`0x7fffffffdda8`	`0x000055555555519f`	Dirección de retorno (nueva cima del *stack*)
`0x7fffffffddb0`	`0x000000000000c000`	Datos de contexto de `convertion`

Estado del stack en este punto:

```
(gdb) x/8x $rsp
0x7fffffffdda8: 0x000055555555519f      0x000000000000c000
0x7fffffffddb8: 0x00000001ffffdde8      0x00007fffffffde14
```

3.6. Ejecución de `ret` (retornar a `convertion`)

Análisis: Retorno al flujo principal (`ret`)

Una vez restaurado el registro base (`RBP`), se ejecuta la instrucción `ret`. Esta operación es la encargada de transferir nuevamente el control del programa a la función llamadora (`convertion`), retomando la ejecución justo después de la instrucción `CALL`.

Comportamiento de la instrucción `ret` en 64 bits:

- **Extracción de la dirección de retorno:** Toma el valor de 64 bits ubicado en la cima actual de la pila (`[rsp]`) y lo carga en el puntero de instrucción (`RIP`).
- **Movimiento del RSP:** Al realizar esta extracción (un *pop* implícito), el registro `RSP` se incrementa automáticamente en 8 bytes, limpiando así la dirección de retorno de la pila lógica.
- **Restauración del estado inicial:** Como resultado, el *stack frame* queda exactamente en las mismas condiciones en las que se encontraba antes de invocar a la subrutina `convert`.

Dirección	Contenido	Descripción
`0x7fffffffddb0`	`0x000000000000c000`	Datos de contexto de `convertion` (Cima restaurada)
`0x7fffffffddb8`	`0x00000001ffffdde8`	Continuación del marco original

Estado del stack en este punto:

```
(gdb) x/8x $rsp
0x7fffffffddb0: 0x000000000000c000      0x00000001ffffdde8
0x7fffffffddc0: 0x00007fffffffde14      0x00007fffffffde10
```

4. Conclusión

El presente trabajo evidencia cómo el diseño basado en una arquitectura multicapa facilita la construcción de soluciones robustas y eficientes. La selección estratégica de lenguajes, de acuerdo con su nivel de abstracción, resulta fundamental para optimizar tanto el rendimiento computacional como la mantenibilidad del código. Asimismo, la integración de Python, C y ensamblador (NASM) no solo maximizó el desempeño en las operaciones matemáticas, sino que proporcionó una comprensión exhaustiva sobre la interacción entre las aplicaciones de alto nivel y la arquitectura de hardware subyacente.

5. Bibliografía

Referencias

[1] Carter, Paul A. *PC Assembly Language*. Noviembre 2006. [Documento en línea]. Disponible para el estudio de la arquitectura x86.

[2] Lu, H. J., Matz, M., et al. *System V Application Binary Interface - AMD64 Architecture Processor Supplement*. [Documento en línea]. Referencia fundamental para la convención de llamadas y paso de parámetros en 64 bits (x86_64).

[3] The NASM Development Team. *NASM - The Netwide Assembler Manual*. [En línea]. Documentación oficial para el ensamblaje de código en arquitecturas multiplataforma.

[4] Stallman, R., Pesch, R., Shebs, S., et al. *Debugging with GDB: The GNU Source-Level Debugger*. Free Software Foundation. [En línea]. Manual de referencia para la inspección de memoria y registros.

[5] The World Bank Group. *World Bank Open Data API Documentation*. [En línea]. Documentación oficial para la consulta y extracción de datos a través de servicios web RESTful.

[6] Real Python. *Python Web Applications with Flask*. [En línea]. Guía y documentación práctica para el despliegue de aplicaciones web y creación de APIs con Python.